CSE557 · BIG DATA ANALYTICS · CIE-1

Expected Questions & Answers

53 high-probability questions with model answers — 2, 3 and 5 markers for each unit, written the way you should write them.

1 **Unit 1 · Big Data & Analytics**
6 × 2-mark · 6 × 3-mark · 6 × 5-mark

2 **Unit 2 · Hadoop, HDFS, MapReduce, YARN**
6 × 2-mark · 6 × 3-mark · 6 × 5-mark

3 **Unit 3 · Spark (CIE-1 part)**
6 × 2-mark · 6 × 3-mark · 5 × 5-mark

Built from your lecture slides, their sample questions and the prescribed textbooks

CIE-1 Edition

Read this first

⚠️ **HONEST NOTE**

No one can promise which questions will appear. These are the **highest-probability** ones: questions taken straight from your slides' **"Sample Questions"** (marked ★ from slides), the topics your teacher spent the most slides on, and the classic questions for each syllabus topic. Learn these well and you can also answer any rewording of them.

How each answer is sized

Marks	Length	What to write
2 MARKS	3–4 lines	A crisp definition + one key point or example. No diagram needed.
3 MARKS	3 points / ~6 lines	Three clear points (or a 3-row table). A small diagram if it helps.
5 MARKS	¾ – 1 page	**Definition → diagram → 4–6 numbered points → example.** The grey "marks split" line under each answer shows where the marks usually go.

🎯 **EXAM-HALL TIPS**

- Start long answers with the **exact definition** — it earns the first mark immediately.
- **Draw the diagram early** in a 5-mark answer and label every box and arrow; number the arrows for step-based topics.
- Use **headings and numbered points**, not paragraphs — examiners scan for keywords.
- A 3-mark "differentiate" question → answer as a **3-row table**.
- Don't over-write 2-markers — save time for the 5-markers.

Question index

Unit 1 · Big Data & Big Data Analytics **3**

2-mark questions (Q1–Q6) 3

3-mark questions (Q7–Q12) 4

5-mark questions (Q13–Q18) 6

Unit 2 · Data Analytics with Hadoop **12**

2-mark questions (Q19–Q24) 12

3-mark questions (Q25–Q30) 13

5-mark questions (Q31–Q36) 15

Unit 3 · Apache Spark Overview **21**

2-mark questions (Q37–Q42) 21

3-mark questions (Q43–Q48) 22

5-mark questions (Q49–Q53) 24

UNIT 1

Big Data & Big Data Analytics

Most-tested areas

Types of digital data · techniques for unstructured data · Big Data definition and Vs · BI vs Big Data · what BDA is · classification of analytics · data scientist · SMP/MPP · shared nothing · CAP theorem.

2 MARKS Short answers Q1 – Q6

Q1. Define Big Data. 2M

According to Gartner, **"Big Data is high-volume, high-velocity and high-variety information assets that demand cost-effective, innovative forms of information processing for enhanced insight and decision making."** In short, it is data that is big in **volume, velocity and variety** — beyond what traditional systems can store and process.

Q2. Why is an email placed in the "unstructured" category? ★ from slides 2M

The **body of an email** is free-form text that follows **no data model** — it can contain sentences, slang, links, images and attachments in any order — so it cannot be stored in rows and columns or queried easily by a program. Hence it is unstructured. (Only its header — To, From, Date, Subject — has some structure, which is why email is sometimes called semi-structured.)

Q3. What is semi-structured data? Give two examples. 2M

Semi-structured data **does not conform to a data model but has some structure**; it uses **tags / label–value pairs** to separate elements, so it is called **self-describing**. Examples: **XML** (used by SOAP web services) and **JSON** (used by REST services, stored natively by MongoDB). HTML and emails are other examples.

Q4. Expand and explain ACID. 2M

ACID are the properties of a transaction supported by an RDBMS: **Atomicity** — the transaction happens fully or not at all; **Consistency** — the database moves from one valid state to another; **Isolation** — each transaction runs as if it were the only one; **Durability** — once committed, changes are permanent even after a crash.

Q5. Differentiate veracity, volatility and variability. 2M

Veracity — trustworthiness of data (noise, bias, abnormality); **Volatility** — how long data remains valid and useful (decides storage duration); **Variability** — data flow is inconsistent, with sudden peaks the system must handle. These are characteristics of data that are **not** definitional traits of Big Data.

Q6. State the CAP theorem. 2M

The CAP (Brewer's) theorem states that in a **distributed system** it is **impossible to provide Consistency, Availability and Partition tolerance at the same time** — at most two can be guaranteed. Consistency: every read gets the latest write; Availability: every request gets a response; Partition tolerance: the system works despite network failures.

3 MARKS Medium answers Q7 – Q12

Q7. Explain the 3 Vs of Big Data. 3M

1. **Volume** — the huge **amount** of data, growing from GB/TB to PB, EB and ZB, coming from internal sources (databases, archives) and external ones (web, social media, sensors).
2. **Velocity** — the **speed** at which data is generated and processed: batch → periodic → near real time → real time (e.g., fraud check at card swipe).
3. **Variety** — the many **types** of data: structured (RDBMS), semi-structured (XML, JSON) and unstructured (text, images, audio, video).

Q8. Differentiate traditional Business Intelligence (BI) and Big Data. 3M

Point	Traditional BI	Big Data
Storage & scaling	All data in a **central server**; scales **vertically**	Data in a **distributed file system**; scales **horizontally**
Analysis mode	Data analysed **offline**	Analysed in **real time and offline**
Data & processing	**Structured** data; data is taken to the processing functions	**Variety** of data; processing functions are taken to the data

Q9. A book is issued from the library. What details about the book can be placed in an RDBMS table? ★ from slides 3M

Structured details go into related tables:

- **Book**(BookID PK, Title, Author, Publisher, ISBN, Edition, YearOfPublication, Category)
- **Member**(MemberID PK, Name, Department, Phone)
- **Issue**(IssueID PK, BookID FK, MemberID FK, IssueDate, DueDate, ReturnDate, Fine)

Constraints: BookID UNIQUE and NOT NULL, DueDate > IssueDate, Fine ≥ 0. The tables are related through BookID and MemberID.

Q10. Differentiate SMP and MPP. 3M

Feature	SMP (Symmetric Multiprocessor)	MPP (Massively Parallel Processing)
Memory & OS	Identical processors share **one common main memory** and a **single OS**	Each processor has its **own memory and own OS**
Coupling	**Tightly coupled**; caches connected by a system bus	Loosely coupled; communicate by **messaging**
Programming	Easier	Harder — program must be divided into communicating segments

Q11. What is a shared nothing architecture? State its advantages. 3M

Of the three multiprocessor architectures — shared memory, shared disk and shared nothing — **shared nothing** is the one where **neither memory nor disk is shared**: each node has its own processor, memory and disk and communicates over the network (Hadoop uses it). Advantages:

1. **Fault isolation** — a fault in one node is confined to that node and exposed only through messages (or their absence).
2. **Scalability** — with a shared disk, nodes must synchronise and take turns, which limits how many can be added; with nothing shared there is no central bottleneck, so nodes can be added freely.

Q12. Explain any three challenges with Big Data. 3M

1. **Huge growth of data** — data grows exponentially, making it hard to separate useful data from noise.
2. **Data retention** — deciding how long to store data; some stays useful for long-term decisions, some becomes irrelevant quickly.
3. **Capture, storage, processing & security** — traditional databases cannot capture, store, search, transfer and secure data that is very large, fast and varied.

Others you may write: cloud computing & virtualisation · lack of skilled professionals · data visualisation.

5 MARKS Long answers Q13 – Q18

Q13. List and explain the classification of digital data with examples.

★ from slides 5M

Digital data is classified into **three types**:

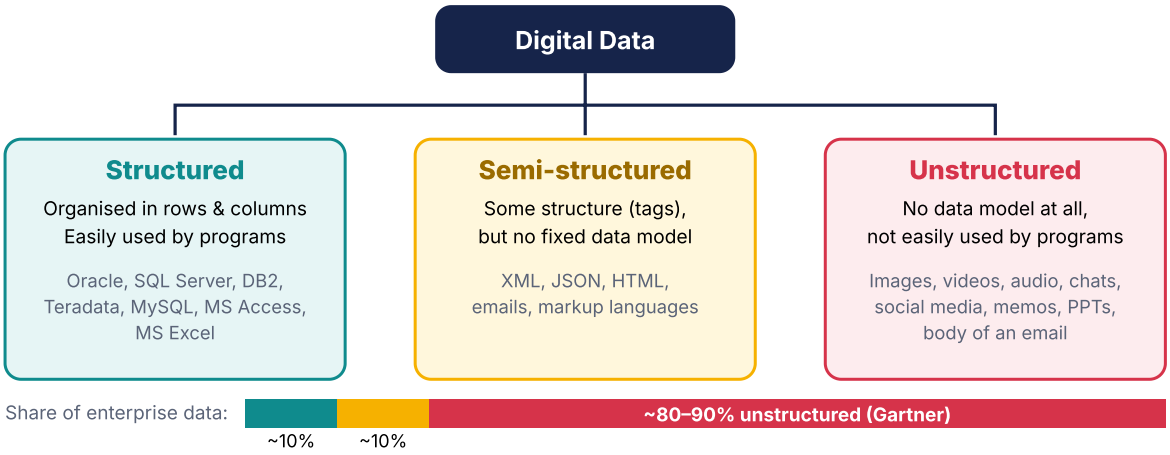


Figure 1 — Classification of digital data

1. **Structured data** — data in an **organised form (rows and columns)** that is easily used by a program; relationships exist between entities. Stored in RDBMS (Oracle, MySQL, SQL Server, DB2), MS Excel, MS Access. Easy to insert/update/delete, secure, indexed, scalable and supports ACID transactions. *Example:* an Employee table (EmpNo, Name, DeptNo).
2. **Semi-structured data** — does **not conform to a data model but has some structure**; uses tags to separate semantic elements and build hierarchies; no separation of data and schema; **self-describing**. *Examples:* XML, JSON, HTML, email.
3. **Unstructured data** — does **not conform to any data model** and cannot be easily used by a program; **80–90%** of an organisation's data. Machine-generated (satellite images, CCTV, radar) or human-generated (emails, documents, chats, social media). *Examples:* images, videos, audio, memos, body of an email.

Comparison: moving from structured → semi-structured → unstructured, **flexibility and scalability increase**, while robustness, transaction support and query power decrease.

Marks split: diagram 1 · three types with definition + example 3 · comparison 1

Q14. Explain the techniques used to find patterns in or interpret unstructured data. ★ from slides 5M



Figure 2 — Techniques for dealing with unstructured data

1. **Data mining** — discovering useful patterns in large data ("knowledge discovery in databases"), using statistics, AI, ML and databases. Algorithms: **association rule mining** (bread buyers also buy butter), **regression** (predict marks from study hours), **collaborative filtering** (recommend what similar people like).
2. **Text analytics / text mining** — patterns from text such as emails, blogs and reviews: text categorisation, clustering, **sentiment analysis**, concept/entity extraction.
3. **Natural Language Processing (NLP)** — enabling computers to understand human language (translation, chatbots, spam detection).
4. **Noisy text analytics** — extracting information from messy text with spelling mistakes, abbreviations and slang ("u coming tmrw??") — chats, SMS, call-centre notes.
5. **Manual tagging with metadata** — a person adds tags (data about data) so the content can be organised and searched (Sports, Cricket, India).
6. **Part-of-speech (POS) tagging** — marking the grammatical role of each word ("Rahul–noun plays–verb cricket–noun").
7. **UIMA** (Unstructured Information Management Architecture) — an **open-source platform by IBM** for real-time content analytics; identifies entities such as organisations and locations.

Marks split: diagram 1 · any 6–7 techniques explained 4

Q15. What is Big Data Analytics? Explain with a neat diagram. 5M

Big Data Analytics is the process of examining large and varied data to uncover patterns, trends and insights that help an organisation make **better and faster decisions** (large data → analyse → patterns → decisions). Example: Amazon analyses past purchases and browsing history to recommend products.

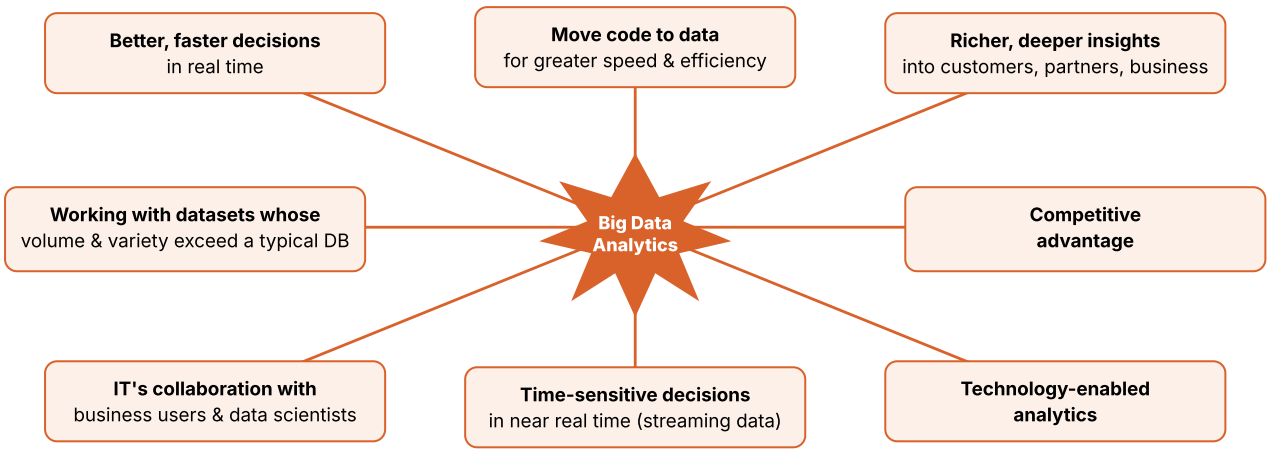


Figure 3 — What is Big Data Analytics?

1. **Technology-enabled analytics** — tools from IBM, Tableau, SAS, R and WPS process huge data.
2. **Richer, deeper insights** — into customers, partners and operations: what customers buy and what else can be sold to them.
3. **Competitive edge** — quicker, better decisions than competitors.
4. **A tight handshake between three communities** — IT (infrastructure), data scientists (analysis) and business users (decisions).
5. **Working with datasets whose volume and variety exceed** traditional databases.
6. **Moving code to data** — the small program is sent to where the huge data is stored, instead of moving terabytes to the program; also time-sensitive decisions in near real time.

Marks split: definition 1 · diagram 1 · 5–6 points 3

Q16. Explain the classification of analytics (Analytics 1.0, 2.0 and 3.0). 5M

There are two schools of thought. **First:** basic analytics (reporting on historical data), operationalised analytics (woven into business processes), advanced analytics (forecasting with predictive and prescriptive models), monetised analytics (earns direct revenue). **Second:** Analytics 1.0, 2.0 and 3.0:

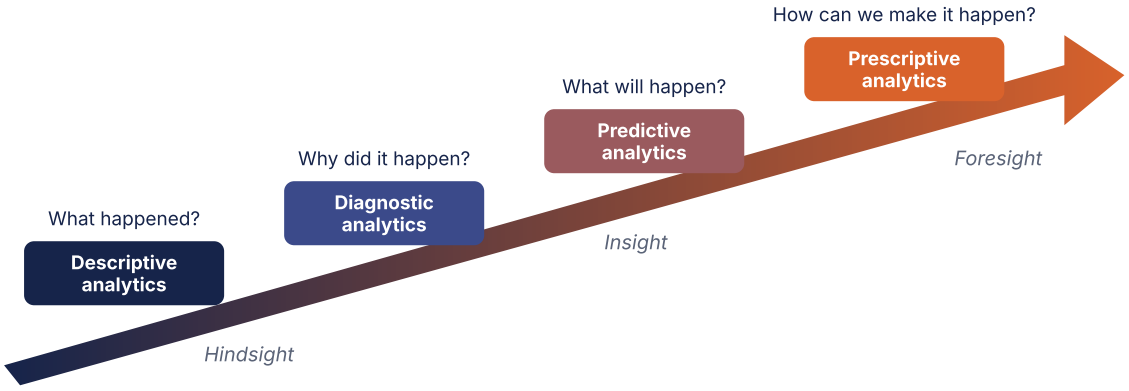


Figure 4 — Descriptive → diagnostic → predictive → prescriptive: hindsight → insight → foresight

	Analytics 1.0	Analytics 2.0	Analytics 3.0
Era	Mid-1950s – 2009	2005 – 2012	2012 – present
Type	Descriptive (what happened?)	Descriptive + predictive (what will happen?)	Descriptive + predictive + prescriptive (what should we do?)
Data	Small, structured, internal; data warehouses, ERP, CRM	Big, mostly unstructured, often external; Hadoop clusters	Blend of big and traditional data, internal + external
Technology	Relational databases	Database appliances, Hadoop, SQL-to-Hadoop	In-memory, in-database, machine learning, agile methods
Example	"Sales fell last month"	"Sales may rise next month"	"Increase stock next month"

Marks split: first school 1 · diagram 1 · 1.0/2.0/3.0 explained 3

Q17. Explain the CAP theorem with an example. 5M

The **CAP theorem** (Brewer's theorem) states that in a distributed computing environment it is impossible to provide all three guarantees — **Consistency, Availability and Partition tolerance** — simultaneously; at best two can be achieved.

- **Consistency** — every read fetches the last write.
- **Availability** — reads and writes always succeed; every non-failing node responds in reasonable time.
- **Partition tolerance** — the system keeps functioning when a network partition occurs.

Example: a coordinator updates your 3:00 PM training with administrator Raju. You ask administrator Rahul, whose copy wasn't updated: "no training". Two answers for the same data = **inconsistent**. Forcing them to sync before every reply makes users wait (less **available**); if they can't talk at all, the system must still work (**partition tolerance**).

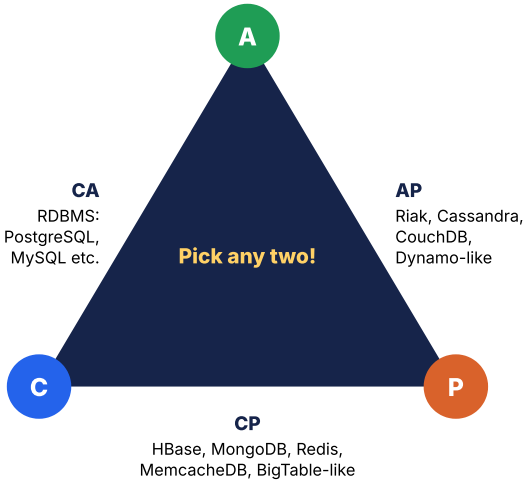


Figure 5 — Databases and CAP

Combinations: CA — traditional RDBMS (MySQL, PostgreSQL) · **CP** — HBase, MongoDB, Redis, MemcacheDB, BigTable · **AP** — Riak, Cassandra, CouchDB, Dynamo. Choose C for bank balances, A for social feeds.

Marks split: statement 1 · C, A, P 1.5 · diagram 1 · example + combinations 1.5

Q18. What is data science? Explain the skills and responsibilities of a data scientist. 5M

Data science is the science of **extracting knowledge from data** — finding hidden patterns using mathematics, statistics, IT, machine learning and pattern recognition. A **data scientist** is the person who applies it to solve real-world problems.

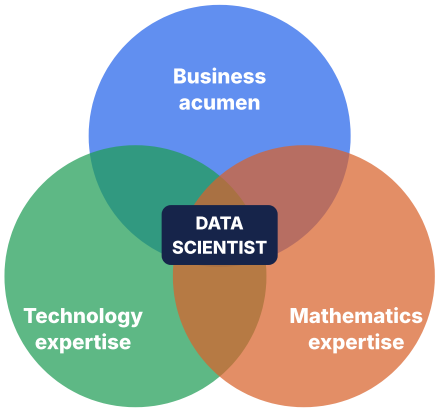


Figure 6 — A data scientist combines three skill sets

Skills:

- **Business acumen** — domain understanding, business strategy, problem solving, communication, presentation, inquisitiveness.
- **Technology expertise** — RDBMS, NoSQL, programming (Python, Java), data warehousing, data mining, visualisation tools.
- **Mathematics expertise** — statistics, algorithms, machine learning, pattern recognition, NLP.

Responsibilities:

1. **Data management** — prepare and integrate large, varied datasets (clean errors, remove duplicates, handle missing values).
2. **Analytical techniques** — apply models and algorithms to find relationships, trends and patterns.
3. **Business / domain expertise** — give results real-world context ("low attendance → early mentoring").
4. **Communication & presentation** — explain findings clearly to stakeholders through charts and simple language.

Marks split: definition 1 · diagram/skills 2 · four responsibilities 2

UNIT 2

Data Analytics with Hadoop

Most-tested areas

MapReduce with the weather example · HDFS design · NameNode/DataNode · file read & write · Federation & HA · classic MapReduce vs YARN job run · failures · schedulers · shuffle and sort.

2 MARKS Short answers Q19 – Q24

Q19. What is Hadoop? Name its core components. 2M

Hadoop is an **open-source framework**, written in Java (by Doug Cutting and Mike Cafarella), that allows us to **store and process large data sets in a parallel and distributed manner**. Core components: **HDFS** (storage layer), **MapReduce** (processing layer) and **YARN** (resource management layer), plus Hadoop Common utilities.

Q20. What is data locality optimisation? 2M

Hadoop tries to run each **map task on the node where its input data (HDFS block) already resides**, so that valuable cluster network bandwidth is not used. If that node is busy, it uses a node in the **same rack** (rack-local); only rarely a node in another rack (off-rack, with an inter-rack transfer).

Q21. What is the default HDFS block size? Why is it so large? 2M

The default HDFS block size is **128 MB** (64 MB in older versions), much larger than a disk block (512 bytes). Large blocks keep the **time spent seeking small** compared to the time spent transferring data, and they reduce the **metadata the NameNode keeps in memory**. A file smaller than a block uses only its actual size on disk.

Q22. What is the role of the Secondary NameNode? 2M

The Secondary NameNode **periodically merges the namespace image with the edit log** (checkpointing) so the edit log does not grow too large. It runs on a separate machine and keeps a copy of the merged image that can be used if the NameNode fails. It is **not a backup NameNode** and cannot take over automatically.

Q23. What is MRUnit? 2M

MRUnit is a **testing library** for MapReduce, used with **JUnit**, that passes known inputs to a mapper or reducer and checks that the outputs are as expected. **MapDriver** tests a mapper (e.g., weather record → (1950, −11)) and **ReduceDriver** tests a reducer (e.g., (1950, [10, 5]) → (1950, 10)); a mismatch fails the test.

Q24. Expand YARN. Why was it introduced? 2M

YARN = **Yet Another Resource Negotiator** (MapReduce 2). Classic MapReduce hit **scalability bottlenecks around 4,000 nodes** because the single JobTracker did both job scheduling and task monitoring. YARN splits these into a **ResourceManager** (cluster resources) and a per-application **ApplicationMaster** (job lifecycle).

3 MARKS Medium answers Q25 – Q30

Q25. Explain the design of HDFS. 3M

HDFS is a file system designed for storing very large files with streaming data access patterns, running on clusters of commodity hardware.

1. **Very large files** — hundreds of MB, GB or TB; clusters store petabytes.
2. **Streaming data access** — built on **write-once, read-many-times**; reading the whole dataset quickly matters more than the latency of the first record.
3. **Commodity hardware** — runs on cheap hardware where node failure is common, and keeps working without noticeable interruption.

Not suited for: low-latency access (use HBase), lots of small files (NameNode memory), multiple writers / arbitrary modifications.

Q26. Differentiate NameNode and DataNode. 3M

Point	NameNode	DataNode
Role	**Master** — manages the file system namespace	**Worker** — the workhorse of the file system
Stores	File tree and **metadata**; persisted as **namespace image + edit log**	The actual **data blocks**
Work / failure	Tells where blocks are; its loss makes the file system unusable (SPOF)	Stores/retrieves blocks when told; sends **periodic block reports** to the NameNode

Q27. What is HDFS Federation? 3M

1. The NameNode keeps every file and block reference in memory, so on very large clusters **memory limits scaling**.
2. **HDFS Federation** (Hadoop 2.x) lets a cluster scale by **adding NameNodes, each managing a portion of the namespace** — e.g., one for `/user`, another for `/share`.
3. Each NameNode manages a **namespace volume** (metadata + a **block pool**). Volumes are independent: NameNodes don't communicate, and one failing doesn't affect the others.

Q28. Explain HDFS High Availability. 3M

1. **Problem:** the NameNode is a **single point of failure**; a cold restart (load namespace image, replay edit log, collect block reports to leave safe mode) can take **30+ minutes**.
2. **Solution:** a pair of NameNodes in **active–standby** configuration; if the active fails, the standby takes over without significant interruption.
3. **Changes needed:** a **shared edit log** on highly available storage; DataNodes send **block reports to both** NameNodes; clients handle **failover** transparently; the standby takes periodic checkpoints (replacing the Secondary NameNode).

Q29. Differentiate the Fair Scheduler and the Capacity Scheduler. 3M

Point	Fair Scheduler	Capacity Scheduler
Goal	Give **every user a fair share** of the cluster over time	Share capacity between **queues with fixed capacities**
Grouping	Jobs placed in **pools** (one per user by default)	Hierarchical **queues**; **FIFO (with priorities)** inside each queue
Behaviour	Lone job gets whole cluster; short jobs finish while long ones run; **preemption** of over-share pools	Like a separate cluster per team/organisation; suited to strict capacity limits

Q30. Explain the failure modes in classic MapReduce. 3M

1. **Task failure** — user code throws a runtime exception, or the child JVM exits suddenly; the TaskTracker marks the task attempt as failed and frees the slot (Streaming: non-zero exit code).
2. **TaskTracker failure** — it crashes or runs slowly and stops sending heartbeats; the JobTracker removes it from its pool. It can be **blacklisted** if more than four tasks of a job fail on it and its faults are well above the average.
3. **JobTracker failure** — the most serious: it is a **single point of failure** with no recovery mechanism, so the job fails (fixed in YARN).

5 MARKS Long answers Q31 – Q36

Q31. Explain how MapReduce finds the maximum temperature for each year, with a data-flow diagram. **5M**

MapReduce breaks processing into two phases — **map** and **reduce** — each with **key–value pairs** as input and output; the programmer writes the map and reduce functions. The NCDC weather dataset is semi-structured and record-oriented (temperature stored as °C × 10, 9999 = missing).

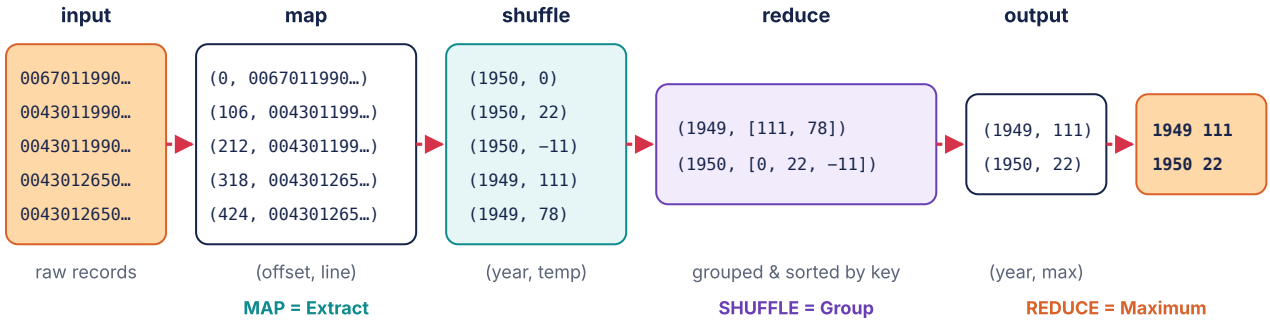


Figure 7 — MapReduce logical data flow

1. **Input:** each line is given to the mapper as (key = line offset, value = record).
2. **Map:** the mapper extracts the **year** and **air temperature**, filters out missing/bad readings and emits (year, temperature): (1950, 0), (1950, 22), (1950, −11), (1949, 111), (1949, 78).
3. **Shuffle & sort:** the framework groups and sorts values by key: (1949, [111, 78]), (1950, [0, 22, −11]).
4. **Reduce:** the reducer iterates through each list and picks the maximum: (1949, 111), (1950, 22).
5. **Java:** `Mapper<LongWritable, Text, Text, IntWritable>`, `Reducer<Text, IntWritable, Text, IntWritable>`, and a **Driver** that sets input/output paths, mapper/reducer classes and output types, then calls `waitForCompletion()`. Output file: `part-r-00000`.

Marks split: definition 1 · diagram 1.5 · map/shuffle/reduce steps 2 · code/classes 0.5

Q32. Explain the anatomy of a file read in HDFS. 5M

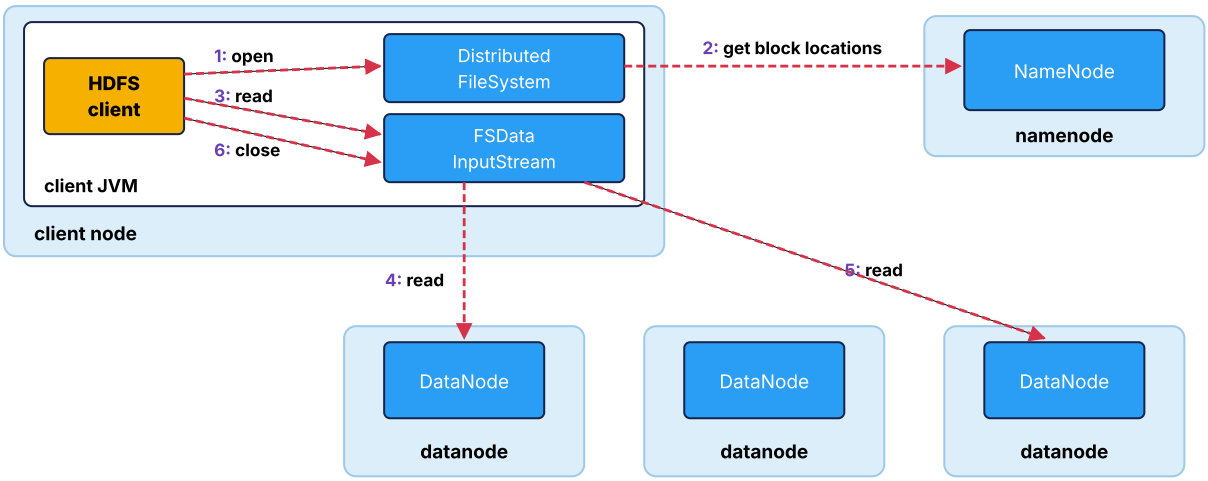


Figure 8 — A client reading data from HDFS

1. The client opens the file by calling `open()` on the **FileSystem** object, an instance of **DistributedFileSystem**.
2. DistributedFileSystem calls the **NameNode using RPC** to get the locations of the first few blocks; for each block, the NameNode returns the DataNodes holding a copy, **sorted by proximity to the client**. It returns an **FSDataInputStream** (wrapping a DFSInputStream).
3. The client calls `read()`; DFSInputStream connects to the **closest DataNode** for the first block.
4. Data is **streamed** from the DataNode to the client as it repeatedly calls read().
5. At the end of a block, DFSInputStream closes the connection and finds the **best DataNode for the next block**, asking the NameNode for further block locations as needed.
6. When finished, the client calls `close()`.

Failure handling: if a DataNode fails, the stream tries the **next closest** one and remembers the failed node; it verifies **checksums** and, on a corrupt block, reads another replica and **reports the corrupt block to the NameNode**. The client reads directly from DataNodes, so the NameNode is not a bottleneck.

Marks split: diagram 1.5 · six steps 3 · failure handling 0.5

Q33. Explain the anatomy of a file write in HDFS. 5M

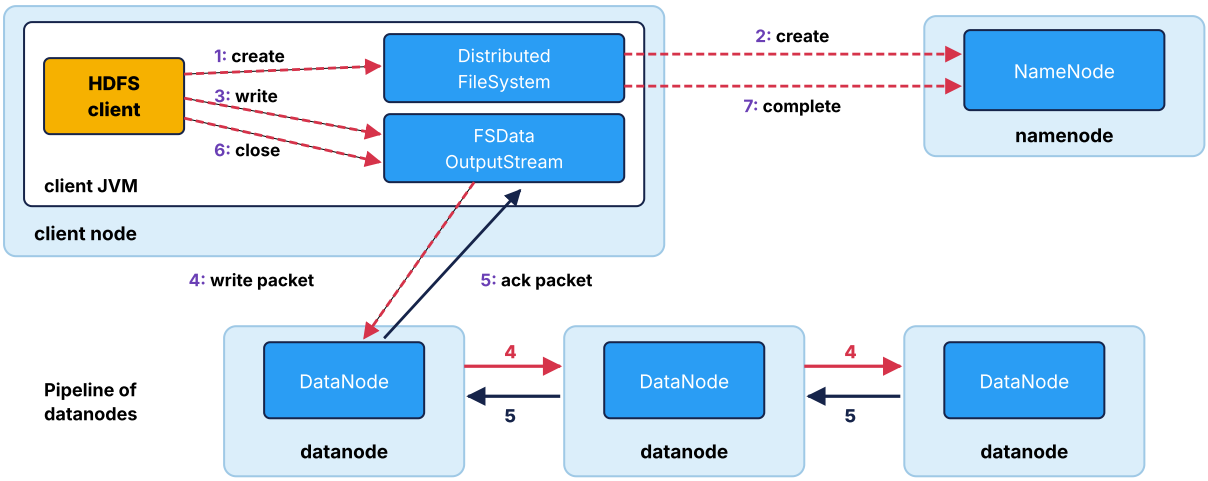


Figure 9 — A client writing data to HDFS

1. The client creates the file by calling `create()` on **DistributedFileSystem**.
2. DistributedFileSystem makes an **RPC call to the NameNode** to create a new file with no blocks. The NameNode checks that the file **doesn't already exist** and the client has **permission**; otherwise an **IOException** is thrown. An **FSDataOutputStream** is returned.
3. As the client writes, **DFSOutputStream** splits data into **packets** placed in the **data queue**. The **DataStreamer** asks the NameNode to allocate new blocks on suitable DataNodes, which form a **pipeline** (three nodes for replication 3).
4. The DataStreamer sends packets to the first DataNode, which stores and **forwards** them to the second, which forwards to the third.
5. Packets wait in the **ack queue** and are removed only when **acknowledged by all DataNodes** in the pipeline.
6. The client calls `close()`, which flushes remaining packets and waits for acknowledgements...
7. ...before telling the NameNode the file is **complete**.

If a DataNode fails: the pipeline is closed, ack-queue packets are put back at the front of the data queue, the current block on good nodes gets a **new identity** (so the partial block is deleted later), the failed node is removed and the **pipeline is rebuilt** from the remaining good DataNodes.

Marks split: diagram 1.5 · seven steps 3 · failure handling 0.5

Q34. Explain how a MapReduce job runs in classic MapReduce (MapReduce 1). 5M

Four independent entities: the **client** (submits the job), the **JobTracker** (coordinates the job run), the **TaskTrackers** (run the tasks) and the **distributed file system** (shares job files).

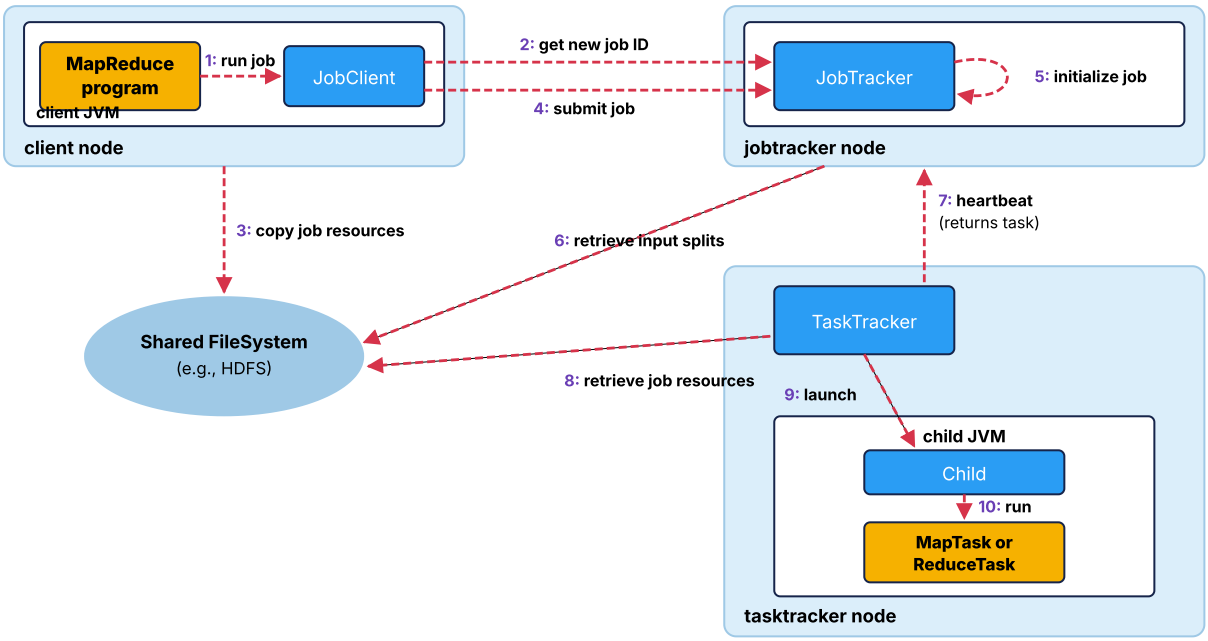


Figure 10 — How Hadoop runs a MapReduce job using the classic framework

1. **Job submission (1–4):** `submit()` creates a JobSubmitter, which asks the JobTracker for a **new job ID**, checks the **output specification** (error if the output directory exists), computes **input splits**, copies the **job JAR** (with high replication), configuration and splits to the shared file system, and calls `submitJob()`.
2. **Job initialisation (5–6):** the job goes into an internal queue; the scheduler creates an object to track it, retrieves the input splits and creates **one map task per split**; the number of reduce tasks comes from `mapred.reduce.tasks`.
3. **Task assignment (7):** TaskTrackers send periodic **heartbeats** saying they are alive and ready; the JobTracker allocates a task in the heartbeat reply.
4. **Task execution (8–10):** the TaskTracker localises the job JAR and distributed-cache files, creates a working directory, and a **TaskRunner launches a new JVM** to run each map or reduce task, so bugs in user code don't affect the TaskTracker.

Marks split: four entities 1 · diagram 1.5 · four phases 2.5

Q35. Explain how a MapReduce job runs on YARN. How is it different from classic MapReduce? 5M

YARN splits the JobTracker's work into a **ResourceManager** and a per-job **ApplicationMaster**. Five entities: **client**, **YARN ResourceManager** (allocates compute resources), **NodeManagers** (launch and monitor containers), **MRAppMaster** (coordinates the job's tasks) and the **distributed file system**.

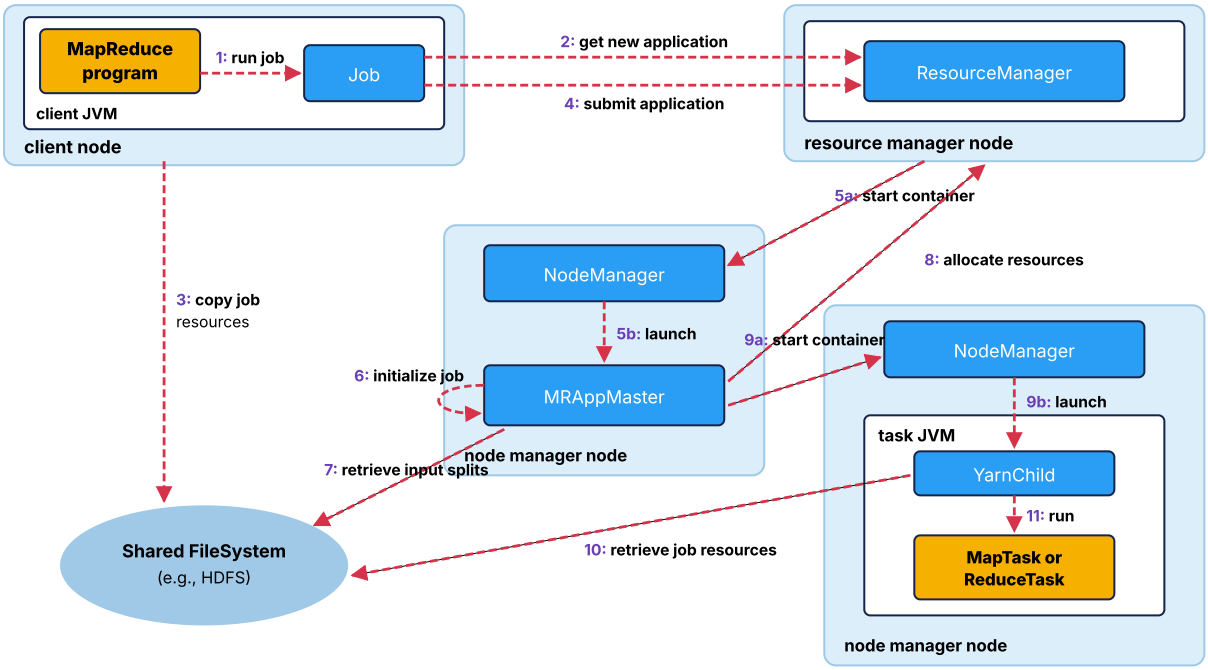


Figure 11 — How Hadoop runs a MapReduce job using YARN

1. **Submission (1–4):** get a new **application ID** from the ResourceManager, check output, compute splits, copy resources to the shared FS, call `submitApplication()`.
2. **Initialisation (5a–7):** the scheduler allocates a container and the RM launches the **MRAppMaster** there via a NodeManager; the AM creates bookkeeping objects and retrieves the input splits.
3. **Task assignment (8):** the AM requests containers from the RM — **map tasks first**, reduce tasks after 5% of maps complete.
4. **Task execution (9a–11):** the AM asks the NodeManager to start the container; **YarnChild** localises the resources and runs the task.

Difference: MR1 has 4 entities and a JobTracker that does both scheduling and monitoring (bottleneck at ~4,000 nodes, single point of failure); YARN has 5 entities, separates resource management from job monitoring, uses flexible containers instead of fixed slots, and scales much further.

Marks split: entities 1 · diagram 1.5 · steps 1.5 · difference 1

Q36. Explain shuffle and sort in MapReduce. 5M

MapReduce guarantees that the **input to every reducer is sorted by key**. The process of performing this sort and **transferring map outputs to the reducers** is called the **shuffle** — the heart of MapReduce.

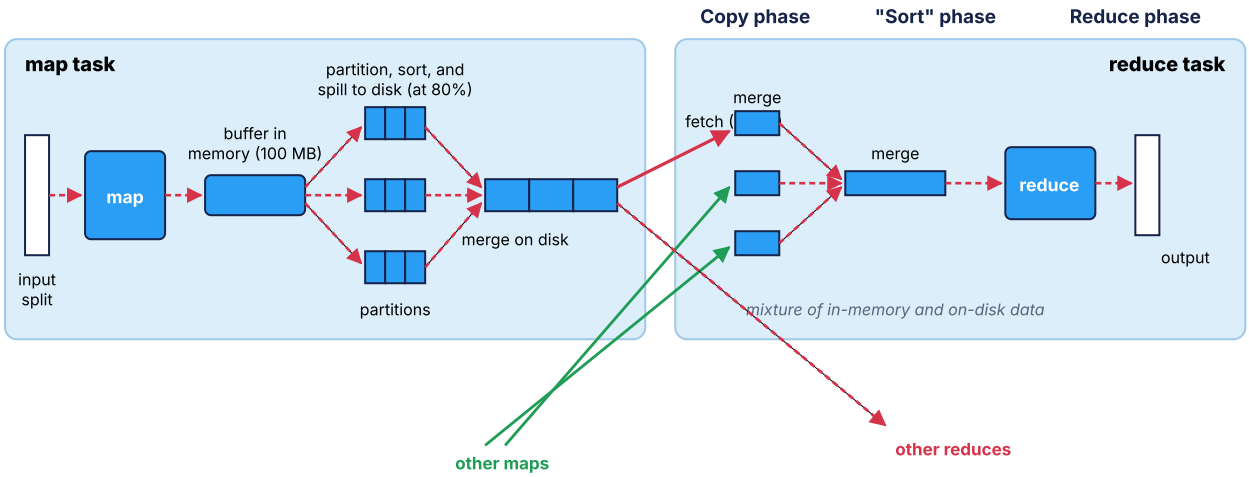


Figure 12 — Shuffle and sort in MapReduce

Map side:

1. Each map task writes output to a **circular memory buffer of 100 MB**.
2. When it reaches the threshold (**80%**), a background thread **spills** it to disk (round-robin into mapred.local.dir).
3. Before writing, data is **partitioned** by the reducer it will go to; within each partition it is **sorted by key in memory** and the **combiner** (if any) runs.
4. Spill files are **merged** into one partitioned, sorted output file (optionally compressed) and served to reducers over **HTTP**.

Reduce side:

1. **Copy phase** — the reducer copies its partition from each map as soon as that map finishes, using **5 copier threads**.
2. **Sort (merge) phase** — map outputs are merged in rounds, keeping the sort order.
3. **Reduce phase** — the reduce function is called once per key; output is written to **HDFS**.

Marks split: definition 1 · diagram 1 · map side 1.5 · reduce side 1.5

UNIT 3 · CIE-1 PART

Apache Spark Overview

Most-tested areas

Spark architecture diagram · components · SparkContext · job/stage/task · run-time flow · DAG & task schedulers · cluster managers & deploy modes · Spark vs MapReduce · installation & first program.

2 MARKS Short answers Q37 – Q42

Q37. What is Apache Spark? 2M

Apache Spark is an **open-source cluster computing framework for data analytics**, designed for quick computation. It supports **in-memory cluster computing** and parallel processing, runs on one or many computers, provides APIs in **Scala, Java and Python**, and is much **faster than Hadoop MapReduce** (up to 100× in memory).

Q38. What is SparkContext? 2M

SparkContext is the **entry point of a Spark application**. It connects to the Spark execution environment and is used to **create RDDs, accumulators and broadcast variables**, access Spark services and **run jobs**. It is created by the driver program (from Spark 2.0 it is wrapped by SparkSession).

Q39. What is an executor in Spark? 2M

An executor is a **process launched for an application on a worker node**. It **runs the tasks** sent by the driver, **keeps data in memory or disk storage** (cache) across them, and reads/writes data to external sources. Every Spark application has its own executors, each running as a separate Java process.

Q40. What is the Spark Shell? 2M

The Spark Shell is a Spark application written in **Scala** that provides an interactive **command-line environment with auto-completion**. It is used to explore Spark's features before writing a standalone application; `sc` (SparkContext) is created automatically. The Python equivalent is `pyspark`.

Q41. List the cluster managers supported by Spark. 2M

A cluster manager **allocates resources across applications**. Spark supports: **Standalone** (Spark's own cluster manager), **Hadoop YARN**, **Apache Mesos**, and **Kubernetes** (Spark 2.3+); **local mode** runs everything on a single machine for testing.

Q42. Differentiate client mode and cluster mode. 2M

In **client mode** the **driver runs on the machine that submits the application** — used for interactive work and shells; closing the terminal stops the app. In **cluster mode** the **driver runs inside the cluster** (on YARN, inside the ApplicationMaster) — used for production; the client can disconnect.

3 MARKS Medium answers Q43 – Q48

Q43. Define task, job and stage in Spark. 3M

1. **Task** — the **smallest unit of work**, sent to a single executor; each partition of an RDD gets its own task.
2. **Job** — a **parallel computation made up of multiple tasks**, created in response to an **action** such as `count()` or `collect()`.
3. **Stage** — a **set of tasks within a job that depend on each other**; a job is split into stages because some steps must wait for others to finish (e.g., a shuffle).

Application ⊃ Job ⊃ Stage ⊃ Task.

Q44. List the responsibilities of SparkContext. 3M

SparkContext, the entry point of a Spark application, is responsible for:

1. Checking the **current status** of the Spark application.
2. **Cancelling** a job or a stage.
3. Running a job **synchronously or asynchronously**.
4. **Accessing or unpersisting a cached RDD**.
5. Programmable **dynamic allocation** of resources.

Q45. Differentiate Apache Spark and Hadoop MapReduce. 3M

Point	Hadoop MapReduce	Apache Spark
Processing	**Disk-based**; writes intermediate results to disk	**In-memory**; up to 100× faster
Execution	Fixed map → shuffle → reduce; batch only	Optimised **DAG** of stages; batch, iterative, interactive, streaming
APIs & libraries	Mainly Java; separate tools (Hive, Pig)	Scala, Java, Python, R, SQL; built-in SQL, Streaming, MLlib, GraphX

Q46. Explain the role of the driver program. 3M

1. The driver is a process that **runs the main() function** of the application and **creates the SparkContext**; it is the **central coordinator** of the master–slave architecture.
2. It handles **job scheduling and negotiation with the cluster manager**, and **stores RDD information**; it converts code into a DAG, stages and tasks and sends tasks to executors.
3. It can run in **client or cluster mode**; it prepares the user code and ships the application (JAR or Python files) to the executors, then monitors them until `stop()`.

Q47. Explain the internal (master–slave) working of Spark. 3M

1. Like Hadoop, Spark uses a **master–slave architecture** with one master and many slaves; the **driver is the central coordinator**.
2. The driver and each **executor run in their own Java process**; one driver handles a large number of executors.
3. A **Spark application is a collaboration of the driver and its executors**. Resources come from Spark's own **standalone cluster manager** or from open-source managers like **YARN** and **Mesos**.

Q48. What is a DAG? Explain the roles of the DAG Scheduler and the Task Scheduler. 3M

1. A **DAG (Directed Acyclic Graph)** is a graph of operations whose arrows only move forward (no loops); Spark records a program's operations as a DAG and **optimises** it before running.
2. When an action creates a job, the **DAG Scheduler** builds the operator DAG and **splits it into stages of tasks**, submitting each stage when it is ready.
3. The **Task Scheduler launches the tasks via the cluster manager** onto executors on the worker nodes.

5 MARKS Long answers Q49 – Q53

Q49. Explain the Apache Spark architecture with a neat diagram. 5M

Spark follows a **master–slave architecture**: one **driver** (central coordinator) and many **executors**, with a **cluster manager** allocating resources. A Spark application is the collaboration of a driver and its executors, each running in its own Java process.

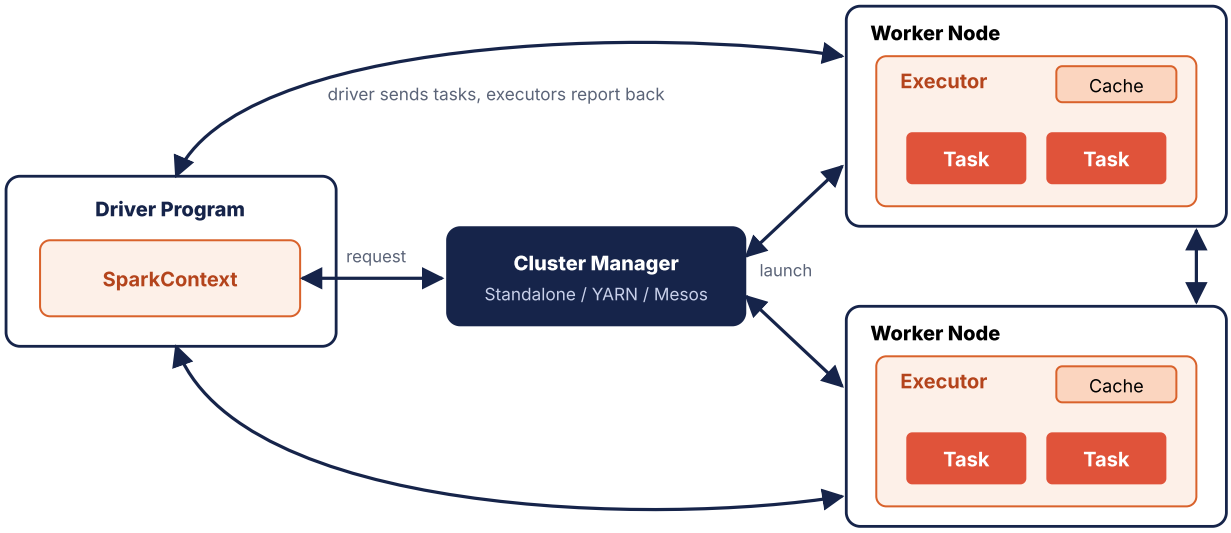


Figure 13 — Apache Spark architecture overview

1. **Driver program** — runs the **main()** function and creates the **SparkContext**; performs job scheduling, negotiates with the cluster manager, stores RDD information; converts code into tasks; runs in client or cluster mode.
2. **SparkContext** — the entry point that connects the driver to the cluster and coordinates the application.
3. **Cluster manager** — **acquires/allocates resources** from the cluster across applications: Standalone, Hadoop YARN or Apache Mesos.
4. **Worker node** — a slave node that runs application code.
5. **Executor** — a process on a worker node that **performs all data processing** (runs tasks) and **stores results in memory (cache)**; reads/writes external data.
6. **Task** — a unit of work sent by the driver to one executor.

Marks split: intro 0.5 · diagram 1.5 · driver, cluster manager, worker, executor, task 3

Q50. Explain the components of Apache Spark. 5M

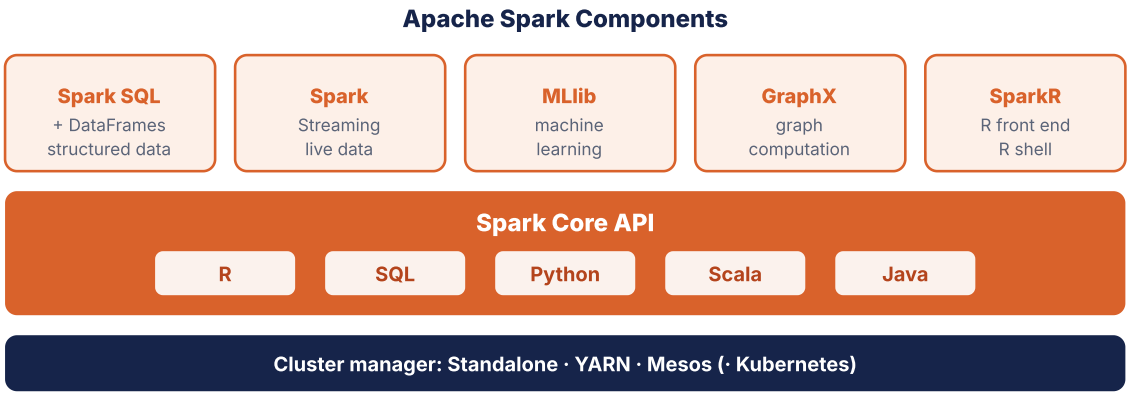


Figure 14 — Apache Spark components

1. **Spark Core** — the **foundation** everything else runs on; provides the execution platform for every Spark application (task scheduling, memory management, fault recovery, interaction with storage) and APIs in R, SQL, Python, Scala and Java.
2. **Spark SQL** — runs **SQL and HiveQL** queries on structured and semi-structured data, often faster than older systems; provides DataFrames.
3. **Spark Streaming** — processes **live data** by breaking it into small **micro-batches** that run on Spark Core (e.g., live logs, tweets).
4. **MLlib** — Spark's **machine learning library**, built to use in-memory processing for iterative algorithms.
5. **GraphX** — a **graph computation engine** for graph-structured data at scale.
6. **SparkR** — a lightweight **front end for R** users to analyse large datasets interactively from the R shell.

All of these run on a cluster manager (Standalone, YARN, Mesos).

Marks split: diagram 1.5 · six components 3.5

Q51. Explain the run-time architecture of a Spark application. 5M

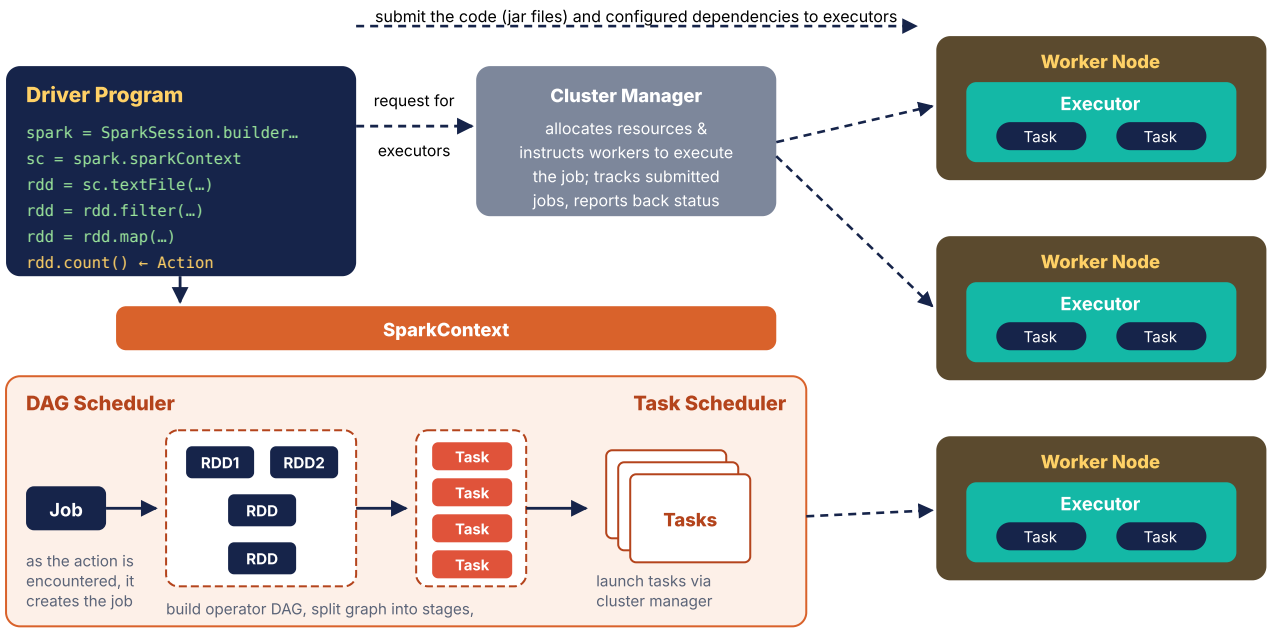


Figure 15 — Internals of job execution in Spark

1. The **driver talks to the cluster manager** and negotiates for resources.
2. The **cluster manager launches executors** on behalf of the driver.
3. The **executors register** themselves with the driver.
4. When a job enters (an action is called), the driver **converts the code into a logical DAG**.
5. The DAG is **optimised**.
6. It is converted into a **physical execution plan** with a set of **stages** (DAG Scheduler).
7. Small execution units called **tasks** are created and **sent to the cluster** (Task Scheduler).
8. **Executors execute** all the tasks assigned by the driver, while the **driver monitors** them.
9. The driver **schedules future tasks** by tracking the location of cached data (data placement).
10. When `stop()` is called on SparkContext, it **terminates all executors** and **releases the resources** from the cluster manager.

Marks split: diagram 1.5 · ten steps 3.5

Q52. Explain the different cluster manager types in Spark. 5M

The **cluster manager** allocates resources (CPU, memory) across Spark applications by launching executors; it does not schedule tasks. It is chosen with `--master` in `spark-submit`.

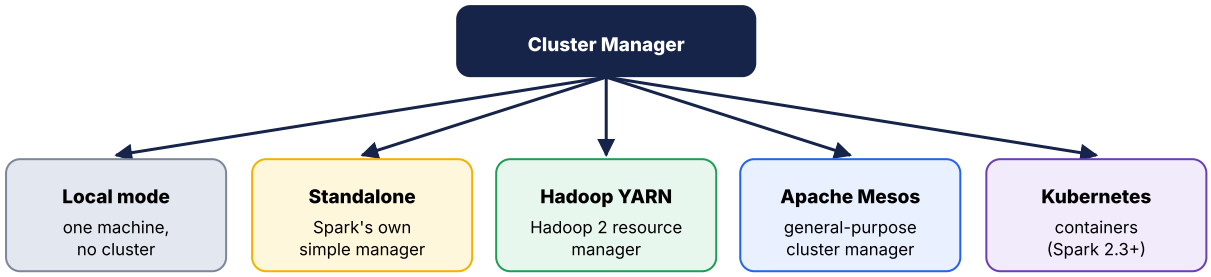


Figure 16 — Cluster manager types

1. **Standalone** — Spark's **own simple cluster manager**, shipped with Spark; easy to set up a cluster on an empty set of machines (one master + workers). `spark://HOST:7077`.
2. **Hadoop YARN** — the **resource manager of Hadoop 2**; runs Spark on an existing Hadoop cluster, sharing resources and reading HDFS; executors run in YARN containers. `--master yarn`.
3. **Apache Mesos** — a **general-purpose cluster manager** that can run Spark alongside other frameworks. `mesos://HOST:5050`.
4. **Kubernetes** — runs the driver and executors as **pods** (Spark 2.3+). `k8s://…`.
5. **Local mode** — no cluster; everything on one machine using `local`, `local[K]` or `local[*]` threads — for learning and testing.

Deploy modes: client (driver on the submitting machine) vs **cluster** (driver inside the cluster).

Marks split: role 1 · diagram 1 · Standalone, YARN, Mesos (+ others) 2.5 · deploy modes 0.5

Q53. Explain the steps to install Apache Spark and write your first Spark program. 5M

Installation (Ubuntu):

1. Install **Java (JDK)** and Python 3: `sudo apt install openjdk-11-jdk python3`.
2. **Download** a pre-built Spark package (`spark-x.y.z-bin-hadoop3.tgz`) from spark.apache.org.
3. **Extract** it (`tar -xzf …`) and move it to `/opt/spark`.
4. Set **environment variables** in `~/.bashrc`: `SPARK_HOME=/opt/spark`, add `$SPARK_HOME/bin` and `sbin` to `PATH`.
5. **Verify**: run `spark-shell --version` or `pyspark` (Web UI at localhost:4040). Alternatively `pip install pyspark`.

First Spark program (word count in PySpark):

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("WordCount").master("local[*]").getOrCreate()
sc = spark.sparkContext
counts = (sc.textFile("input.txt")
            .flatMap(lambda line: line.split())   # split lines into words
            .map(lambda w: (w, 1))                # (word, 1)
            .reduceByKey(lambda a, b: a + b))     # add counts per word
print(counts.collect())                           # action → job runs
spark.stop()
```

Run it with `spark-submit --master local[4] wordcount.py`. Shell version: `sc.textFile("README.md").count()`. Bundled example: `./bin/run-example SparkPi 10` → "Pi is roughly 3.14".

Marks split: installation steps 2.5 · program 2 · running it 0.5